

Software Engineering for CS

Week 6

INSTRUCTOR : DR. NATASH ALI MIAN

School of Computer and Information Technology

Beaconhouse National University

Software Errors – Importance

- History of software
 - Softwares encountered notorious bugs that were the cause of financial lose and deaths of many people.

AT & T Bug

- Famous bugs are
- AT & T long distance breakdown bug 1990
 - ill placed break statement in the code,
 - caused the 1/3rd of entire American network to go down for 9 hours.
 - Loses to banks, for ATM and transactions

Pentium Bug

- Pentium bug
 - software error in microcode of Pentium microprocessor, which resulted in error of floating point calculations problems.
 - Intel had to take back all the Pentiums, and it caused huge financial loss

Major Failures due to Software Bug

Therac-25 Radiation Overdoses (1985-1987): A software bug in the Therac-25 radiation therapy machine caused patients to receive lethal doses of radiation, leading to several deaths and injuries.

Ariane 5 Flight 501 (1996): The Ariane 5 rocket's maiden flight ended in failure due to a software bug in the guidance system, causing the rocket to self-destruct shortly after liftoff.

Patriot Missile Failure in Dhahran (1991): A software bug in the Patriot missile defense system caused it to incorrectly calculate the time, leading to the failure to intercept an incoming Scud missile during the Gulf War, resulting in 28 deaths and many injuries.

NASA Mars Climate Orbiter (1999): The Mars Climate Orbiter mission failed due to a software bug that caused the spacecraft to approach Mars at too low an altitude, leading to its destruction upon entry into the Martian atmosphere.

Knight Capital Group Trading Loss (2012): A software bug in Knight Capital Group's trading algorithm caused the firm to incur a loss of \$440 million in just 45 minutes, nearly bankrupting the company.

Major Failures due to Software Bug

Heartbleed Bug (2014): The Heartbleed bug, a serious vulnerability in the OpenSSL cryptographic software library, exposed sensitive information such as usernames, passwords, and encryption keys, affecting millions of websites and servers worldwide.

Equifax Data Breach (2017): A software vulnerability in Equifax's website allowed hackers to access the personal information of approximately 147 million consumers, including names, Social Security numbers, birthdates, and addresses.

NASA Mars Polar Lander (1999): The Mars Polar Lander mission failed due to a software bug in the spacecraft's landing radar software, which prematurely shut off the descent engines, causing the lander to crash on the Martian surface.

Sony PlayStation Network Outage (2011): A software bug and subsequent cyber attack on Sony's PlayStation Network resulted in the theft of personal information from approximately 77 million user accounts and a 23-day outage of the network.

Major Failures due to Software Bug in Last 5 Years

WhatsApp Outage (2021): A software bug caused a global outage of the WhatsApp messaging service, preventing millions of users from sending or receiving messages for several hours, disrupting communication and causing inconvenience.

Facebook Data Leak (2019): A software bug in Facebook's photo API allowed third-party apps to access the private photos of millions of users without their consent, leading to a significant data privacy breach and regulatory scrutiny.

T-Mobile Network Outage (2020): A software bug in T-Mobile's network caused a widespread outage affecting voice and data services for millions of customers across the United States, disrupting communication and connectivity.

Google Calendar Outage (2019): A software bug in Google Calendar caused a global outage of the service, preventing users from accessing their calendars, scheduling events, and receiving reminders, impacting productivity and causing inconvenience.

Garmin Ransomware Attack (2020): A software vulnerability in Garmin's network infrastructure led to a ransomware attack that encrypted the company's systems and disrupted its services, including GPS navigation, aviation, and fitness tracking, causing widespread service outages and financial losses.

Major Failures due to Software Bug in Last 5 Years

Boeing 737 MAX MCAS System (2018-2019): As previously mentioned, faulty software in the Maneuvering Characteristics Augmentation System (MCAS) contributed to two fatal crashes involving Boeing 737 MAX aircraft, resulting in 346 deaths and the grounding of the entire fleet.

SolarWinds Cyberattack (2020): A software vulnerability in SolarWinds' Orion platform allowed hackers to infiltrate the networks of numerous government agencies and corporations worldwide, including the U.S. government, resulting in one of the most significant cyber espionage incidents in history.

COVID-19 Vaccine Appointment System Failures (2021): Various instances of software bugs and technical issues plagued online appointment scheduling systems for COVID-19 vaccine distribution, causing delays, confusion, and frustration among users in several countries.

Robinhood Trading Outage (2020): A software bug in the Robinhood trading platform led to a widespread outage during a period of high market volatility, preventing users from accessing their accounts and executing trades, resulting in significant financial losses for some investors.

UK National Health Service (NHS) Test and Trace App Issues (2020): The NHS Test and Trace app experienced software bugs and technical glitches, including inaccuracies in contact tracing notifications and compatibility issues with certain smartphones, hindering its effectiveness in combating the spread of COVID-19.

Importance of Software and Impact of Errors

Software is present everywhere microwave oven, cars and software use is expanded.

That means a small software error can cause your microwave oven to explode, where system failure can cause losses more than the system itself.

A software problem can cause life losses.

We have to be careful for the use of such systems, where loss of life is a bigger loss.

Types of Errors

Types of Requirements Errors

Errors of omission

Errors of commission

Errors of clarity and ambiguity

Errors of speed and capacity

Errors of Omission

Errors of omission are most common among requirements errors

Domain experts easily forget to convey domain knowledge to requirements engineers, because they consider that to be obvious and implicit

Errors of Clarity and Ambiguity

Second most common errors are those of clarity and ambiguity

Primarily, because natural languages (like English) are used to state requirements, while such languages are themselves ambiguous

For example: object

Errors of Commission

Errors of commission can also find their way into the requirements documents

Performance Errors (errors of speed and capacity)

Performance, that is errors of speed and capacity, are also found in requirements

Primarily, these occur due to conflicting understanding or competing needs of different stakeholders

Severity Levels

1. Critical (or Blocker) Severity

Definition:

A defect that causes a **complete failure of the system or major functionality**, making it **impossible to proceed with testing or usage**.

Impact:

The system crashes or data is lost.

No workaround is available.

Testing or operations cannot continue until it's fixed.

Examples:

Application does not launch.

Login function fails for all users.

System hangs or crashes on startup.

Action:

Immediate attention required; fix urgently before release or further testing.

2. Major (or High) Severity

Definition:

A defect that **breaks a major part of the functionality**, but **the system is still partially usable**.

Impact:

A key feature fails, but the rest of the application works.

Workarounds may exist but are inconvenient.

Examples:

"Save" button not working on a form.

Incorrect calculations in core business logic.

Unable to generate essential reports.

Action:

Fix should be prioritized; may block major features if not resolved.



3. Moderate (or Medium) Severity

Definition:

A defect that **affects minor functionality or causes incorrect behavior**, but the **main features still work correctly**.

Impact:

The issue does not stop testing or main use.

Workarounds are usually available.

Examples:

Misalignment in a UI element.

Error message displays incorrect text.

Search results show partial but not complete data.

Action:

Fix needed but not urgent; can be scheduled for a later release.



4. Minor (or Low) Severity

Definition:

A defect that has **minimal impact** on functionality or user experience — usually **cosmetic or typographical errors**.

Impact:

System functionality is unaffected.

Only visual or presentation issues.

Examples:

Spelling mistakes in labels or tooltips.

Color mismatch in UI.

Minor alignment or spacing issues.

Action:

Fix when convenient; often deferred to a later version.



Summary

Severity Level	Impact	Example	Fix Priority
Critical	System unusable or crashes	App fails to start	Immediate
Major	Major feature fails	Login works but cannot save data	High
Moderate	Minor feature affected	Incorrect error message	Medium
Minor	Cosmetic or trivial	Spelling error in UI	Low

Other ways to classify Errors (By Sources)

Defects can occur **at any phase** of the SDLC, and each phase has its own **typical causes** or **sources of defects**.

Requirements

Source:

Defects introduced during the **requirements gathering and analysis** stage.

Causes:

Incomplete or ambiguous requirements.

Misunderstanding of customer needs.

Poor communication between stakeholders and analysts.

Missing non-functional requirements (e.g., performance, security).

Examples:

The software doesn't meet user expectations.

A key business rule is not implemented because it wasn't documented.

Prevention:

Conduct requirement reviews and validation sessions.

Use clear, testable, and traceable requirements.

Involve end-users and domain experts early.

Requirements phase Defects

Source:

Defects introduced during the **requirements gathering and analysis** stage.

Causes:

Incomplete or ambiguous requirements.

Misunderstanding of customer needs.

Poor communication between stakeholders and analysts.

Missing non-functional requirements (e.g., performance, security).

Examples:

The software doesn't meet user expectations.

A key business rule is not implemented because it wasn't documented.

Prevention:

Conduct requirement reviews and validation sessions.

Use clear, testable, and traceable requirements.

Involve end-users and domain experts early.

Design Phase Defects

Source:

Defects introduced when converting requirements into system or architectural designs.

Causes:

Incorrect interpretation of requirements.

Poor software architecture or data modeling.

Interface design mismatches between modules.

Lack of design validation or peer review.

Examples:

System designed without scalability in mind.

Wrong data relationships or flow between components.

Prevention:

Conduct design reviews and walkthroughs.

Use modeling tools (e.g., UML diagrams).

Apply design standards and best practices.

Coding (Implementation) Phase Defects

Source:

Defects introduced during **code development** when implementing design.

Causes:

Syntax or logic errors in code.

Poor coding standards or lack of code reviews.

Incorrect algorithm or data structure use.

Misuse of third-party APIs or libraries.

Examples:

Divide-by-zero error.

Incorrect loop condition causing infinite loops.

Null pointer exceptions or memory leaks.

Prevention:

Enforce coding standards.

Perform unit testing and peer code reviews.

Use static code analysis tools.

Testing Phase Defects

Source:

Defects that occur during **test design or execution** — not in the software itself, but in how testing is performed.

Causes:

Poorly designed test cases (missing scenarios).

Incomplete test coverage.

Wrong expected results in test cases.

Inadequate test data.

Examples:

A defect not detected because the test didn't cover an edge case.

False positives/negatives due to incorrect test setup.

Prevention:

Review and validate test cases.

Use traceability matrix to ensure coverage.

Perform regression and exploratory testing.

Deployment (Implementation/Release) Phase Defects

Source:

Defects introduced when **software is moved to production** or deployed in a live environment.

Causes:

Incorrect configuration settings.

Environment mismatch between test and production.

Missing or corrupted files during installation.

Inadequate deployment procedures.

Examples:

Application fails to connect to production database.

Missing dependencies or incorrect version control.

Prevention:

Use deployment checklists and automation tools.

Conduct pilot or beta testing.

Perform environment validation before go-live.

Maintenance Phase Defects

Source:

Defects introduced when **modifying the system post-release**, such as during bug fixes or enhancements.

Causes:

Poor impact analysis before changes.

Regression errors due to updates.

Inadequate documentation or version control.

Examples:

A bug fix causes another module to break.

Old data incompatible with updated code.

Prevention:

Perform regression testing after updates.

Maintain detailed change logs.

Follow proper configuration management practices.

Summary

SDLC Phase	Source of Defects	Common Causes	Example
Requirements	Ambiguous or missing requirements	Miscommunication, incomplete analysis	Feature not aligned with user needs
Design	Poor architecture or design flaws	Incorrect interpretation, interface mismatch	Wrong data flow between modules
Coding	Logic or syntax errors	Lack of code review, poor practices	Infinite loop, null pointer
Testing	Faulty test cases	Incomplete coverage, bad data	Missed edge cases
Deployment	Environment or setup errors	Misconfiguration, version mismatch	App fails in production
Maintenance	Regression or update errors	Inadequate testing after changes	Old features break after fix

Thanks

